ML Offline Design

Where We Stand

Rodrigo Cunha — Principal ML Engineer
Custom Bidding Agent · ML Lane · 2026

What you'll know by the end

- What's already **decided**, and why
- What the offline design is actually **made of**
- What it needs from the team (help to review, please!)
- What's still **open** — and what's blocking it

Part 1

The system today

Today vs. tomorrow

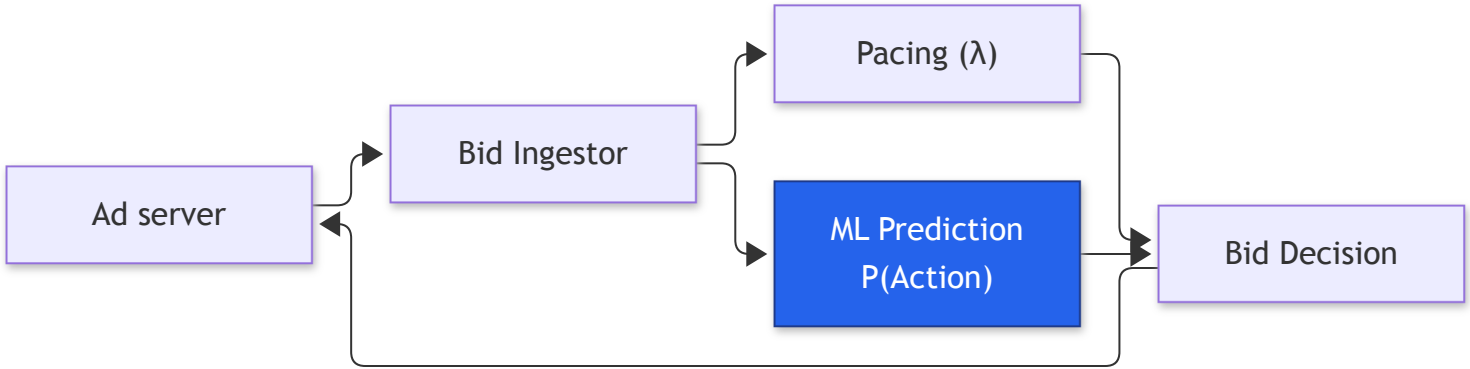
Today – every advertiser bids on awareness:

`Final Bid = λ × Max CPM`

Tomorrow – this project unlocks performance bidding:

`Final Bid = λ × Max CPA × P(Action)`

Four services, one request



our lane: **ML Prediction**

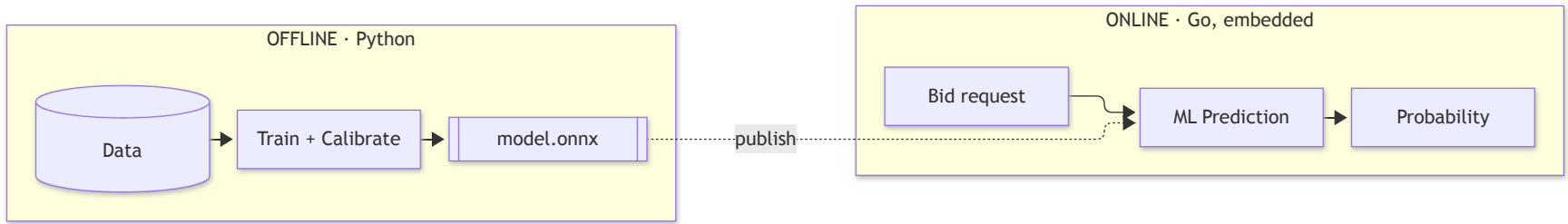
Part 2

What’s already decided

Four decisions, locked in

Decision	Status
ML returns a **probability**, not a dollar value	Settled
Served as **ONNX**, embedded in the Go agent — no separate server	Settled
Calibration is mandatory, versioned with the model	Settled
Served artifact is **one self-contained file** (prep + model + calibration)	Settled

Two worlds, one bridge



ONNX is the only thing that crosses the bridge

The latency budget

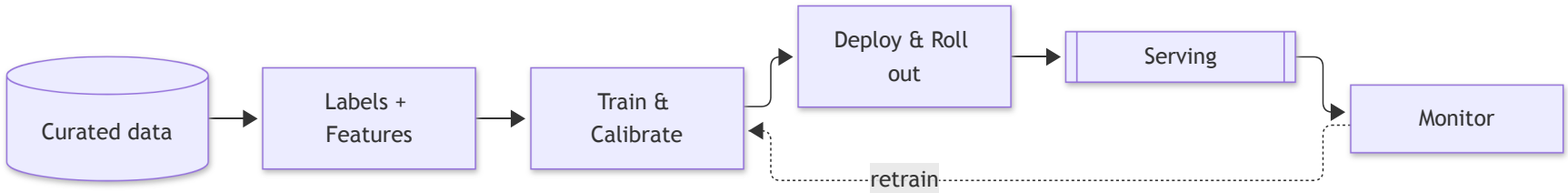
Budget	Whose
10 ms, end-to-end HTTP	the ad server's hard timeout — whole agent
~3 ms, p99	the ML step's own slice — our firm target

a budget inside a budget — ingestor, pacing, and decision share the rest

Part 3

What the offline design is made of

The whole pipeline, end to end



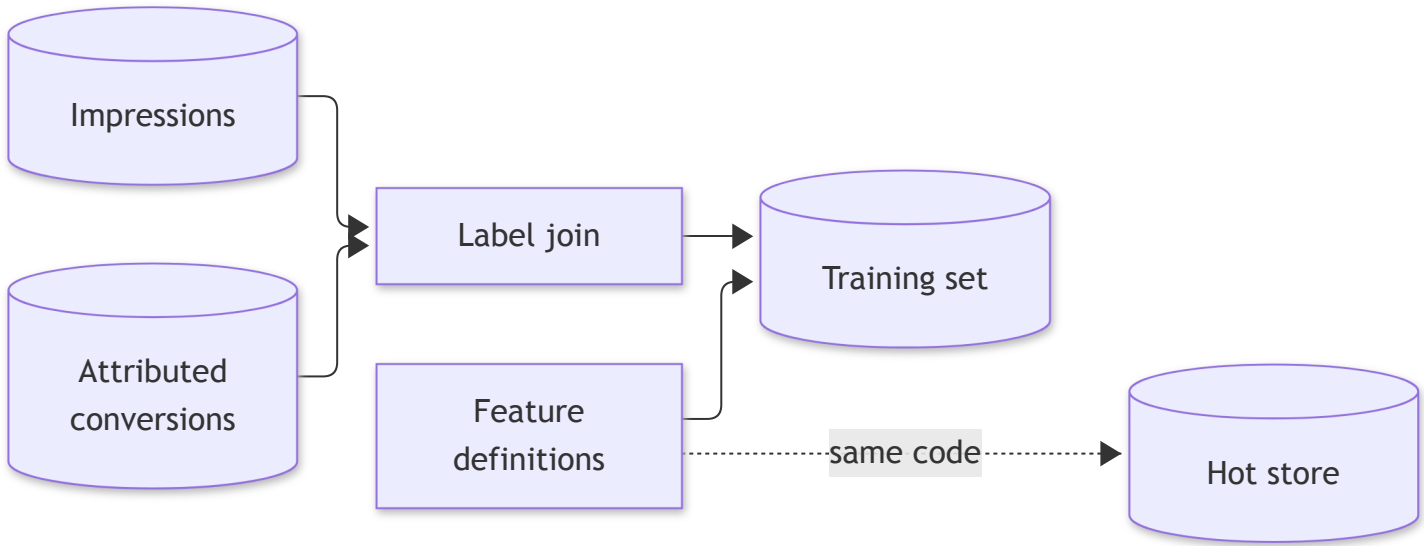
the map for what follows – deploy & roll out gets the most depth

Data in

- Impression-level training data
- Attribution labels (conversions)
- Decisions (from serving)

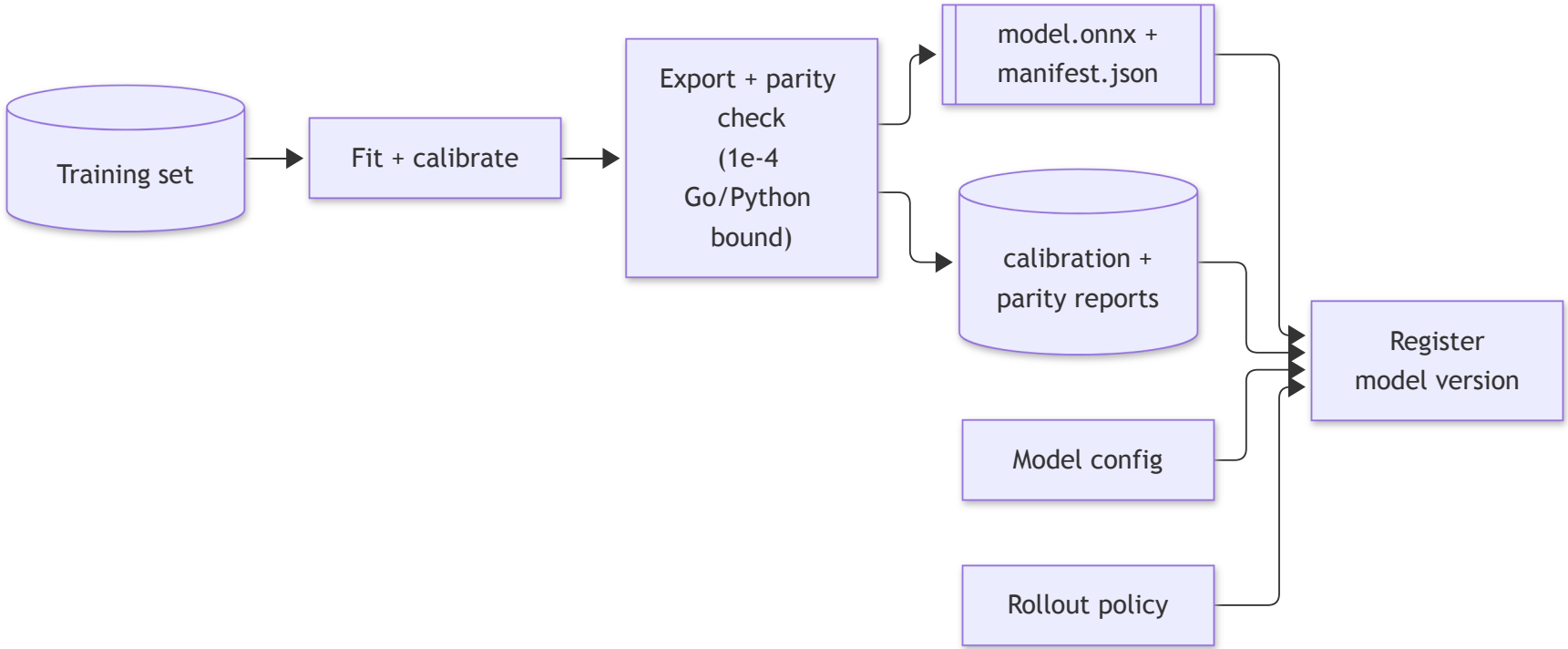
the first two: from the client, requirements fixed, delivery still open · decisions: ours already, via the serving contract

Building a training set



one feature definition feeds both training and serving – no drift

Train & calibrate



everything a model version needs ships together – nothing hand-written can drift from what trained

Two repos, one training pipeline

- **DAG repo** – the pipeline framework + every model's own definition (schedule, model config, **rollout policy**)
- **Workload repo** – one versioned, pipeline-agnostic package (training, calibration, ETL, feature code) + the one deployment job

nothing model-specific lives in the workload repo – that's what keeps it reusable

The training pipeline’s files

inputs from the DAG repo, outputs registered onto the model version

DAG repo — per-model pipeline folder (input)

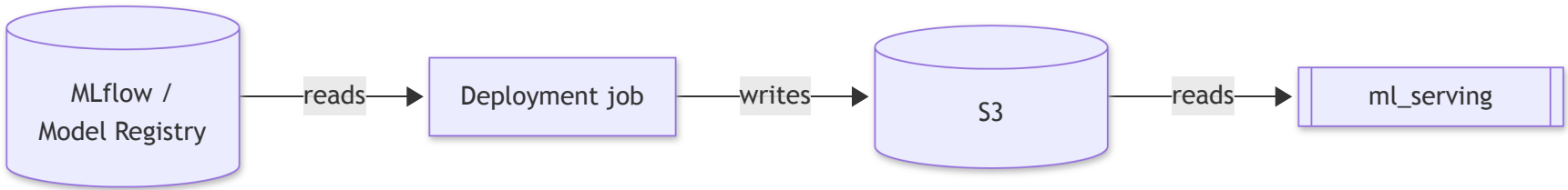
```
pipelines/training_pixel_purchase/
├── dag.py
├── pipeline.yaml          schedule, compute tier
├── training.yaml          model configuration
└── rollout.yaml           rollout policy
```

S3 — model storage (output, once published)

```
<model_name>/
├── v1/model.onnx          trained + calibrated, immutable
├── v1/manifest.json       input schema, feature join spec
├── v2/model.onnx
├── v2/manifest.json
└── rollout_config.json    champion · challenger · previous, mutable
```

rollout policy is authored in the DAG repo, registered with the model version at training time — the deployment job reads it from the registry, not this repo

The deployment job is the bridge



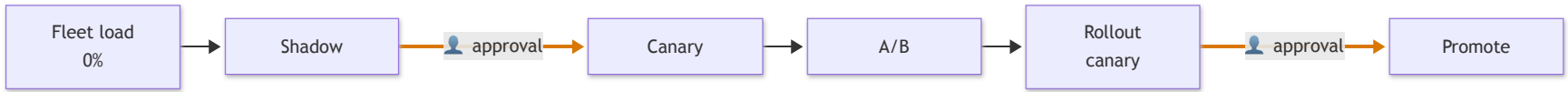
ml_serving only ever talks to S3 – MLflow stays entirely on our side

Model versions: three roles, always

- **Champion** – serving all traffic today
- **Challenger** – moving through the states below
- **Previous** – yesterday's champion, kept for rollback

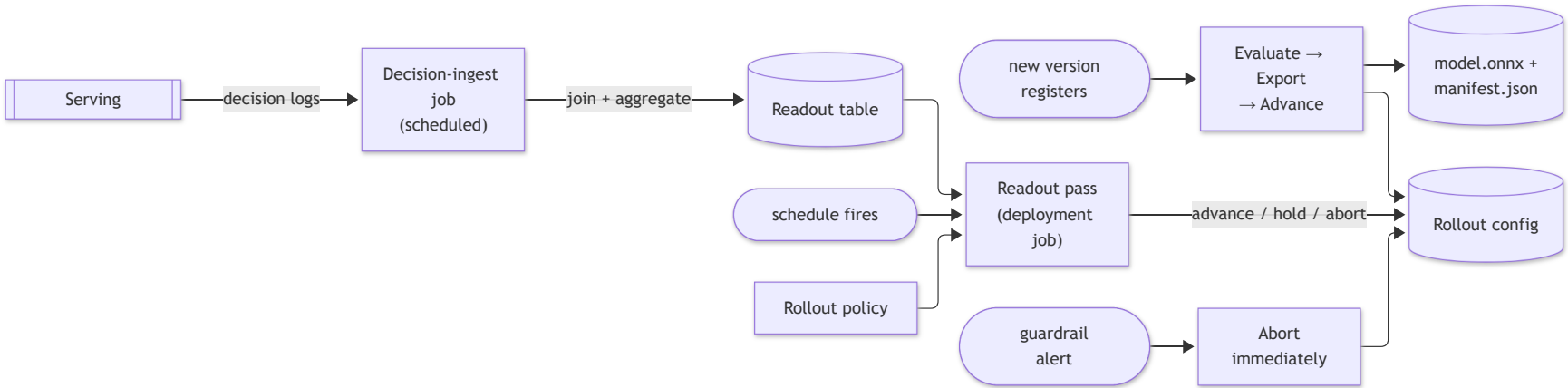


Deploy & roll out



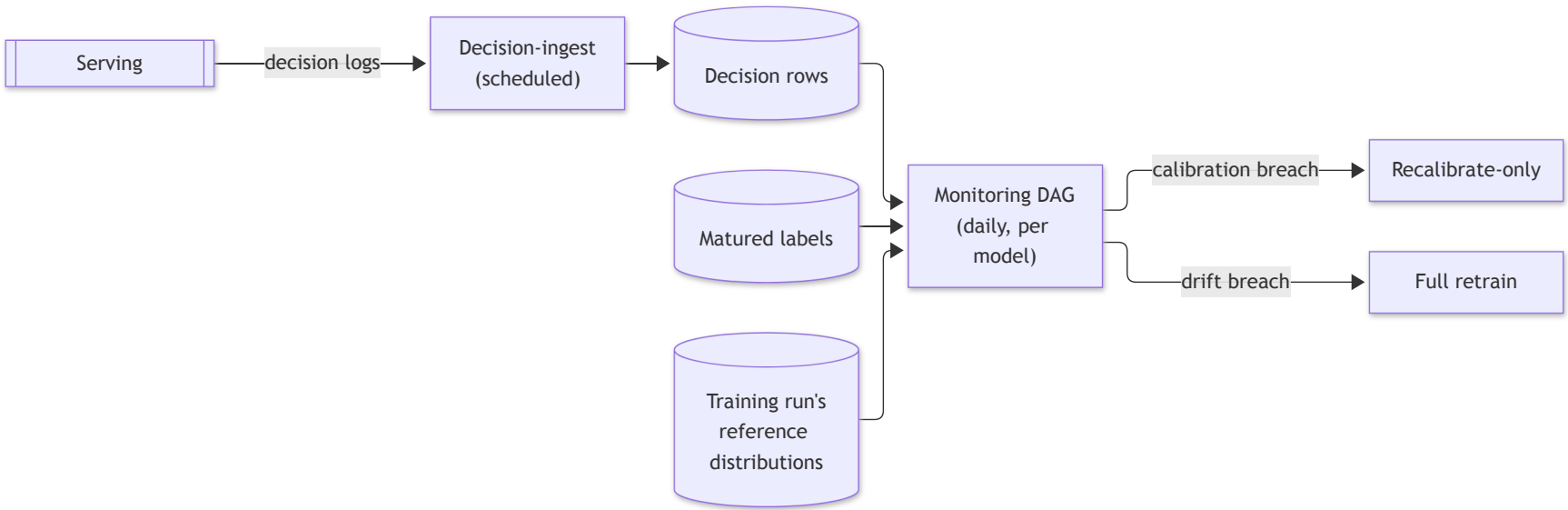
the challenger's path to becoming champion · auto-abort anywhere on a guardrail breach

What drives the rollout



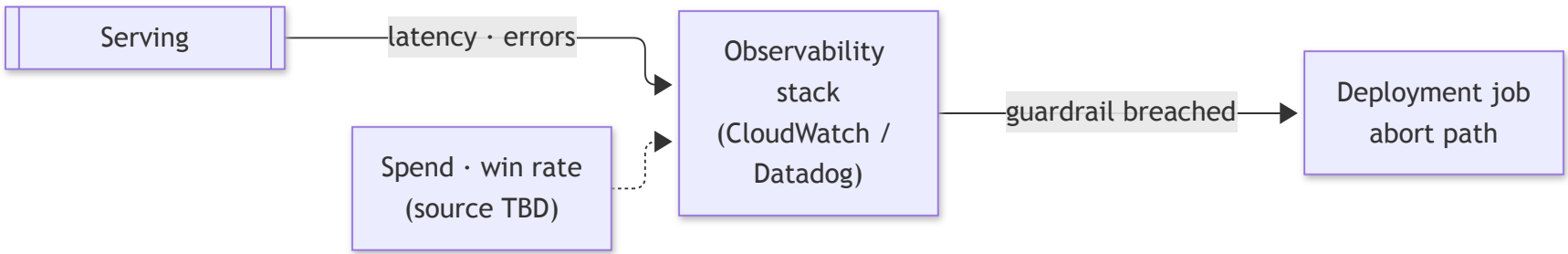
one job, three triggers · nothing here is manual except the two gates

Monitor & retrain



model quality, daily – serving-health guardrails (spend, latency, errors) run continuously in the observability stack, a separate system entirely

How the guardrail alerts work

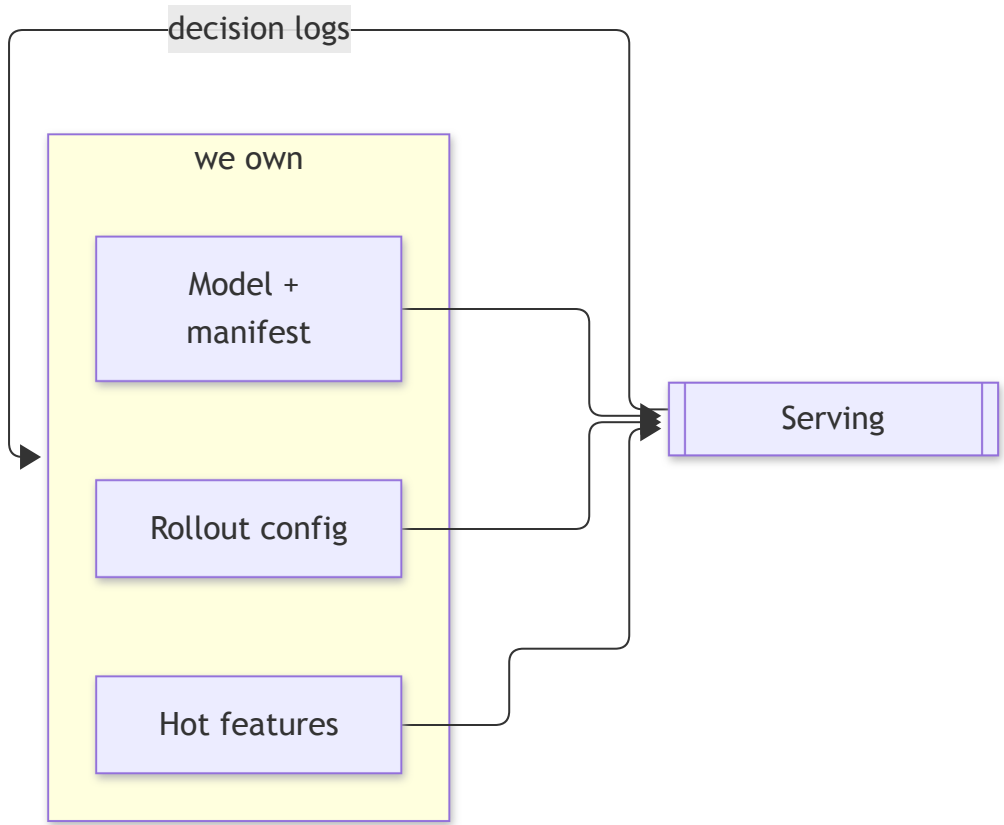


latency + errors: continuous, agent-internal · spend + win rate: guardrails designed, data source still open with the client

Part 4

The contract with serving

Four touchpoints



Who writes, who reads

Touchpoint	We write	Serving reads	Changes on
Artifact + manifest	✓	✓	new model version
Rollout config	✓	✓	new rollout state
Hot-feature store	✓	✓	every 5–10 min
Decision logs	✓ (ingest)	✓ (emits)	every scored candidate

Part 5

Infra this needs

The stack

Layer	Status
Cloud platform	confirmed
ML compute, tracking, model registry	confirmed
Specific tooling choices within it	ours to pick
Our own orchestration instance	ours; reuse-vs-new still open
Fast key-value store for hot features	needed; exact service deferred

no standing compute — pipelines submitted at trigger time only, nothing idle between runs

Part 6

What’s still open

The two real blockers

1. **Attribution window + method** — what exactly we're training on
2. **Cross-boundary data-access mechanism** — tables are named, how they move isn't

both need an answer from the client

Everything else

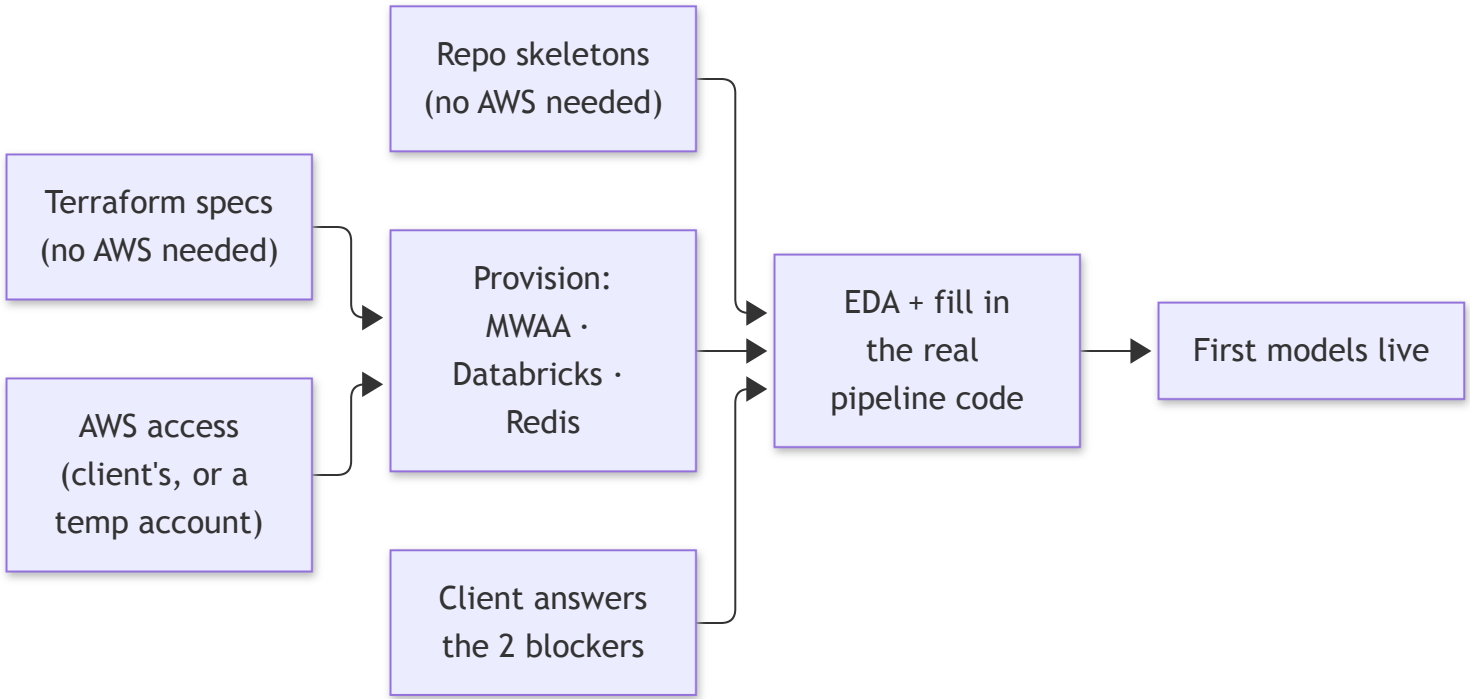
Category	Roughly
Needs the client	~13 items
Settles once EDA runs on real data	~10 items
Ours to decide, no dependency	~7 items
Asks on the serving side	~8 items

full list lives in the design register, not here

Part 7

What happens next

The path forward



two independent blockers, not one – access itself isn't the finish line either

Takeaways

- The design is **complete and concrete** — not a sketch
- **Two client answers, plus AWS access** unlock EDA and the real build
- The contract with serving is **fixed** — both sides build in parallel

Thank you — questions?